Ders 4 Genişletilmiş Notu: Kitap-Yazar-Kütüphane Uygulaması: Kapsülleme, Constructor Overloading, Static ve Finalize

Ders 4 Genişletilmiş Notu: Kitap-Yazar-Kütüphane Uygulaması: Kapsülleme, Constructor Overloading, Static ve Finalize

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. IDE'ler Arasında Proje Taşıma
- 2. Problem: Kitap ve Yazar Nesneleri
- 3. Nesne Yazdırmak, Referanslar ve ==
- 4. Varsayılan Constructor ve İlk Değerler
- 5. String Birleştirme ve Gizli Tip Dönüşümü
- 6. Neden Bu Kod Nesne Yönelimli Değil?
- 7. Yazar Sınıfını Yeniden Yazmak: Constructor, `this`, Get Metotları
- 8. Kitap Sınıfı: Constructor Overloading ve `this(...)`
- 9. `static` Metotlar ve Alanlar
- 10. `private` Set Metodu ve Şase Numarası Örneği
- 11. `finalize()`, Çöp Toplayıcı ve `@Override`
- 12. Kütüphane Sınıfı: Kitap Dizisi, Ekleme, Listeleme
- 13. Klavyeden Kapasite, Nesne Dizisi ve Bellek Düzeni
- 14. Toplu Ekleme: `kitapEkle(Kitap[])` ve Derste Çıkan Hata
- 15. Tekrar Alıştırması: Birden Fazla Yazar
- 16. Dizinin Sonu, `length` ve Dönüş Değerini Kullanmamak
- 17. Sayısal Tipler Arasında Atama

1. IDE'ler Arasında Proje Taşıma

Ders, bir öğrencinin başka bir IDE'de yazdığı projeyi NetBeans'e geçirememesiyle ilgili soruyla başladı. Hoca sorunun kaynağını proje yapılandırmasıyla açıkladı: Her projenin Maven, Ant ya da Gradle gibi bir yapısı ve IDE'ye özgü yapılandırma dosyaları vardır. Bunlar uyumsuzsa proje "paket olarak" geçirilemez. Dosyalar tek tek açılabilir ama proje olarak açılamaz.

Hocanın bildiği kadarıyla NetBeans ile Eclipse birbirinin projesini açabiliyor, fakat IntelliJ'in dosya biçimini tanımıyor. Benzetmesi şuydu: Bu, bir WinRAR dosyasını medya oynatıcıyla açmaya çalışmaktan farksızdır. Program dosyanın biçimini tanımadığı için açamaz.

En basit yol, `src` klasörünün altındaki `.java` dosyalarını alıp NetBeans'te yeni bir proje oluşturmak ve kodu o projenin altına kopyalamaktır. NetBeans'te File > New Project denince Gradle, Maven ve Ant seçenekleri çıkar. Derste basit olduğu için Ant kullanılıyor. Hoca, başka bir IDE'nin projesinin Maven ya da Gradle üzerinden açılabilmesinin denenebileceğini, ama bunu kendisinin denemediğini de ekledi.

Hoca ardından yeni bir proje açtı. Bu bölümde kodu eşzamanlı yazmak gerekmediğini, tanımlamalardan oluştuğu ve algoritmik bir zorluk içermediği için izlemenin yeterli olduğunu söyledi.

2. Problem: Kitap ve Yazar Nesneleri

Hocanın kurduğu senaryo şuydu: Bilgisayarda kitap ve yazar nesneleri olsun. Kitabın ID'si, adı ve yazar bilgisi tutulsun. Şimdilik bir kitabın tek yazarı olduğu kabul edilir. İlişkilerin yönü ve derecesiyle ilgilenilmeyecek, yalnızca iki nesne arasında ilişki kurulacak.

NetBeans'in açtığı varsayılan sınıf refactor ile `Test` diye yeniden adlandırıldı. Hoca bunun yerine "driver" da denebileceğini söyledi. `Kitap` için aynı pakette yeni bir sınıf dosyası oluşturuldu.

> **Ders sonrası not:** Hoca, tek dosyada birden fazla sınıfın "Java'da yapılamadığını" söyledi. Doğrusu: Bir `.java` dosyasında birden fazla üst düzey sınıf olabilir, ama en fazla biri `public` olabilir ve onun adı dosya adıyla aynı olmalıdır. İki sınıflı dosya derlendi, ikinci sınıf da `public` yapılınca "class B is public, should be declared in a file named B.java" hatası alındı (javac ile denendi).

Sınıfa önce alanlar yazıldı:

```
public class Kitap {
    int id;
    String ad;
    Yazar yazar;   // tek yazar kabulü
}
```

Hoca burada `String`'in Java'da büyük S ile yazıldığını, küçük `string` olmadığını hatırlattı. Bazı dillerde (hatırladığı kadarıyla C#'ta) ikisi de vardır.

`Yazar` sınıfı henüz olmadığı için üçüncü alan hata verdi. Hoca bu hatayı öğretici buldu, çünkü bir **bağımlılığı** gösteriyor: Gerçek hayatta bir kitabın yazılması için bir yazarın var olması gerekir. Demek ki senaryoyu kurgularken hangi nesnenin önce var olması gerektiğini düşünmek gerekir. Ardından `Yazar` yazıldı ve hata kendiliğinden kalktı:

```
public class Yazar {
    int id;
    String ad;
}
```

Hoca alan sayısını bilerek az tuttu, çünkü uzatınca kurgulama ve test uzun sürüyor.

3. Nesne Yazdırmak, Referanslar ve ==

Test sınıfında önce bir yazar oluşturuldu:

```
Yazar y1 = new Yazar();
System.out.println(y1);
```

Hoca `new` olmadan nesnenin oluşmadığını, önce `new` işleminin yapıldığını, sonra oluşan nesnenin referansının `y1`'e atandığını vurguladı. NetBeans'te `sout` yazıp Tab'a basmak `System.out.println` satırını üretir.

Çalıştırınca ekranda ID ve ad değil, `javaapplication7.Yazar@...` biçiminde, sınıf adıyla başlayan bir değer görüldü. Hocanın açıklaması, `y1`'in tek bir değer değil, değişik alanlardan oluşan bir yapı olduğu için ekrana referansının yazıldığıydı.

> **Ders sonrası not:** Hoca burada "integer gibi string gibi primitif şeyler" dedi. Doğrusu: `int` primitiftir, `String` ise bir sınıftır (`String.class.isPrimitive()` değeri `false`, javac ile denendi). Ayrıca yazdırılan değer doğrudan bellek adresi değildir: `println`, nesnenin `toString()` metodunu kullanır ve varsayılan sonuç `sınıfAdı@hashCode` biçimindedir. `o.toString()` değerinin `o.getClass().getName() + "@" + Integer.toHexString(o.hashCode())` ile aynı olduğu javac ile denendi.

Sonra alanlara doğrudan değer atandı. Bu aşamada alanlara erişim belirteci yazılmamıştı ve test sınıfı aynı pakette olduğu için bunlara erişebiliyordu:

```
y1.id = 1;
y1.ad = "Ahmet";
```

Hoca `y1.ad = new String("Ahmet");` yazılabileceğini de gösterdi, ama IDE bunu gereksiz bulup uyarı verdi ve doğrudan atamayla devam edildi. Program yeniden çalıştırılınca aynı çıktı görüldü: `y1`'i yazdırmak alanları göstermez.

3.1 == Nesnelerde Neyi Karşılaştırır?

Hoca içeriği aynı ikinci bir yazar oluşturdu ve iki nesneyi karşılaştırdı:

```
Yazar y2 = new Yazar();
y2.id = 1;
y2.ad = "Ahmet";

if (y1 == y2)
    System.out.println("Nesneler aynı");
else {
    System.out.println(y1);
    System.out.println(y2);
}
```

Her ikisinin içeriği `1` ve `Ahmet` olmasına rağmen `else` kolu çalışır, çünkü `==` işleci bu nesneler için içerik karşılaştırması tanımlamaz, referansları karşılaştırır. Bellekte farklı alanlarda tutulan iki ayrı nesne vardır. Bu örnekte sonucun `false` olduğu javac ile denendi.

3.2 Kitaba Yazar Atamak

Ardından bir kitap nesnesi oluşturuldu:

```
Kitap k1 = new Kitap();
k1.id = 1;
k1.ad = "Java 1";
k1.yazar = y1;
```

`Kitap`, kendi içinde `Yazar` türünde bir referans tutar. C'den bildiğimiz gibi `=` işleci aynı ya da birbirine dönüştürülebilen tipler arasında atama yapar. `k1.yazar` ile `y1` aynı tipte olduğu için atama geçerlidir. Hoca bu yapıyı iç içe bir `struct`'a (nested struct) benzetti.

`k1` doğrudan yazdırılınca yine referans görünür. Hoca sonra alanları biçimlendirerek yazdırdı:

```
System.out.println(k1);
System.out.println("[" + k1.id + " " + k1.ad + " " + k1.yazar + "]");
```

Çıktıda `1` ve `Java 1` okunur, `k1.yazar` için ise yine referans görünür. Çünkü `k1.yazar` da referans tipli bir değişkendir.

> **Hocanın vurgusu:** `new` ile oluşturulan bütün değişkenler referans tiplidir. Bir sınıf nesnesini doğrudan yazdırırsak bu tür bir değer görürüz, bunu unutmayalım.

4. Varsayılan Constructor ve İlk Değerler

Hoca bu kez yeni bir kitap nesnesini hiçbir alanına değer vermeden yazdırdı. `new Kitap()` dendiğinde constructor çalışır. Biz constructor tanımlamadıysak varsayılan constructor çalışır ve alanlara tiplerine göre ilk değer verilir: `int` için `0`, `float` için `0.0`, referans tipleri için `null`.

> **Ders sonrası not:** Hoca varsayılan constructor'ı Java Virtual Machine'in çalıştırdığını söyledi. Doğrusu: Varsayılan constructor'ı derleyici üretir. Constructor yazılmamış `Yazar` sınıfı derlenince `javap` çıktısında `Yazar();` satırı görülür (javac ve javap ile denendi).
>
> **Ders sonrası not:** Hoca çıktıyı `0, 0, 0` diye okudu ve ad ile yazar referans tipli olduğu için değerlerinin 0 olduğunu söyledi. Hocanın az önce söylediği kural doğrudur, bu okuma değil: `int` alan `0`, `String` ve `Yazar` alanları `null` olur. Çıktı `0 null null` biçimindedir (javac ile denendi).

5. String Birleştirme ve Gizli Tip Dönüşümü

Soru: `System.out.println` içinde `int` ile `String` topladık. Sonuç `String` mi olur?

Cevap: Evet, `String` olur. Hoca bunu normalde tip dönüşümü konusunda anlatacağını, ama sorulmuşken göstereceğini söyledi. Java'da da C'de olduğu gibi tipler arasında gizli dönüşüm vardır. Hocanın deyişiyle genellikle büyük veri tipleri yanındaki küçük tipleri "yutar". `String`'in yanına `int` ya da `float` gibi bir şey eklersek onu da kendine katar.

```
String x = "AAA" + 3;      // AAA3
x = "AAA" + 3.0;           // AAA3.0
```

Buradaki `+` işleci string birleştirmedir (string concatenation). İkinci işlenen `String`'e dönüştürülebildiği için ifadenin tamamı `String` olur. Hoca değişkenin metodun başında tanımlanmak zorunda olmadığını, `String x`'in araya yazılabildiğini de belirtti.

5.1 `null` Değeri

Java'da referans tipli değişkenlere ilk değer verirken `null` kullanılır, çünkü `0` kullanamayız. Hoca bunu bir yerel değişkenle gösterdi:

```
Yazar ye;
System.out.println(ye);   // derleme hatası
```

Derleyici "variable ye might not have been initialized" hatası verir (javac ile denendi). İlk değer vermek gerekiyorsa `Yazar ye = null;` yazılır ve ekranda `null` görünür.

Asıl şaşırtıcı örnek `null`'un bir `String`'e eklenmesidir:

```
x += "BBB" + null;
```

Hoca, ekrana bakarak ifadenin "hepsini null haline getirdiğini" söyledi.

> **Ders sonrası not:** Sonuç metnin tamamının `null` olması değildir. `null`, `"null"` metni olarak sona eklenir. `x` değeri `AAA3.0` iken bu satırdan sonra `x`, `AAA3.0BBBnull` olur (javac ile denendi).
>
> **Hocanın vurgusu:** `null` ile bir `String` toplanınca sonucun `String` olması çoğu kişiyi şaşırtır. Bazı yerlerde mülakat sorusu olarak sorulur.

Hoca bir de işleç önceliğine dikkat çekti: Önceki ifade zaten `String` olduğu için `null` ona eklenir, ama bir `int` değişkeniyle başlayan ifadede durum farklıdır. Bir `int` değişkeni tanımlayıp önce toplamanın yapıldığını, sonra `String`'in yanına eklendiğini göstermek istedi, fakat örneği tam hatırlayamadı.

> **Ek bilgi (derste anlatılmadı):** `+` işleci soldan sağa işlenir. `String` görülene kadar sayılar toplanır, sonrasında birleştirme yapılır. Kısa nottaki `1 + 2 + "A" + 1 + 2` ifadesinin sonucu `3A12` olur (javac ile denendi).

`String` dışında da aynı mantık geçerlidir: `double` ile `int` toplanır ya da çarpılırsa sonuç otomatik olarak `double` olur. Hoca bunu "büyük tipler küçük tipleri yutar" diye özetledi ve konuya daha sonra döneceğini söyledi.

6. Neden Bu Kod Nesne Yönelimli Değil?

Hoca burada yönü değiştirdi: Nesne yönelimli programlamadan kastımız, yapılacak işlerin mümkün olduğunca soyutlanmasıydı. Oysa test sınıfı şu an yazarın ve kitabın ID'sine, adına doğrudan erişiyor. Test sınıfını programı kullanan kişi gibi düşünelim. Bir program yapıp sattığımızı, örneğin Hepsiburada gibi bir uygulama olduğunu varsayalım. Kullanıcı gidip orada bir kitabın yazarının adını ya da ID'sini değiştiremez. Bu bilgilerin dışarıdan erişilemez olması gerekir.

Erişilemezliği sağlamak için `Yazar` ve `Kitap` sınıflarındaki alanlar `private` yapıldı. Bunun üzerine test sınıfındaki bütün atamalar hata verdi. Hoca o satırların tamamını sildi, çünkü bunlar nesne oluşturuyor olsa da nesne yönelimli metodolojiye aykırıydı. Derleyici bu erişimde "id has private access in Yazar" hatası verir (javac ile denendi).

> **Hocanın vurgusu:** Nesne oluşturmak tek başına nesne yönelimli programlama değildir. Test sınıfında alanlara doğrudan değer atamak bu metodolojiye aykırıdır.

7. Yazar Sınıfını Yeniden Yazmak: Constructor, `this`, Get Metotları

Yazar oluşturulurken ID ve ad bir kez girilir, sonra gerekirse okunur. Set metoduna bile gerek yoktur. O hâlde bir constructor yazmak mantıklıdır. Hoca constructor'ı sınıfın üstüne ya da altına yazmak arasında fark olmadığını, ama genelde yukarıya yazmanın iyi olduğunu söyledi.

İlk deneme parametreyle alanın aynı adda olması sorununu gösterdi: `id = id;` yazınca hangi `id`'nin hangisine atanacağı belli olmuyor, IDE burada bir sorun bildiriyor.

> **Ders sonrası not:** `id = id;` derleme hatası vermez. Parametre kendine atanır, alan değişmez. `new Yazar(7).id` değeri `0` çıktı (javac ile denendi). Bu yüzden `this` gereklidir.

Oluşturulan nesnenin alanına erişmek için `this` kullanılır. `this.` yazınca IDE'nin listesinde yazdığımız alanların yanında hiç yazmadığımız birçok üye görünür. Hoca bunların çoğunun `Object` sınıfından gizli bir kalıtımla geldiğini, ileride bakılacağını söyledi.

Yazarın kendini tanıtması için parametresiz bir `void` metot eklendi. Alanlar `private` olduğu için, okuma gerekirse get metotlarına ihtiyaç var. Hoca önce `getId`'yi elle yazdı, sonra `getAd`'ı NetBeans'in aracına ürettirdi: Alanları seçip sağ tıklayıp Refactor > Encapsulate Fields. Bu pencerede set metotları kapatıldı, alan görünürlüğü `private`, erişim metotlarının görünürlüğü `public` seçildi.

Araç iki şeye dikkat çekti:

- Üretilen `getAd` doğrudan `return ad;` yazdı. Bu yanlış değil, ama hoca sınıfın değişkeni olduğunu açıkça belirttiği için `this.ad` yazmayı daha güzel buluyor.
- Refactor, `yazarTanit` içindeki doğrudan alan erişimlerini de `this.getId()` ve `this.getAd()` çağrılarına çevirdi. Yani metot, nesnenin kendi get metodunu çağırıyor, o da alanı geri döndürüyor.

```
public class Yazar {
    private int id;
    private String ad;

    public Yazar(int id, String ad) {
        this.id = id;
        this.ad = ad;
    }

    public int getId() { return this.id; }
    public String getAd() { return this.ad; }

    public void yazarTanit() {
        System.out.println("[YAZAR ID:" + this.getId()
                + "--->YAZAR AD:" + this.getAd() + "]");
    }
}
```

Artık yazar oluşturma kuralı değişti. IDE, constructor'ın başına bir yorum yazılmışsa onu dokümantasyon olarak da gösterir. Test sınıfında:

```
Yazar y1 = new Yazar(1, "ALI");
y1.yazarTanit();     // [YAZAR ID:1--->YAZAR AD:ALI]
```

Constructor yazıldıktan sonra parametresiz `new Yazar()` da derlenmez (javac ile denendi). Kullanıcı yalnızca yazar bilgilerini görmek isteyebilir, kitaplarla hiç uğraşmadan yazarlara göre kayıt aratabilir. Bu yüzden `yazarTanit` metodunu test sınıfından çağırmak doğrudur.

Hoca IDE'nin alanları `final` yapma önerisini de gösterdi. `final` yapılırsa değer değiştirilemez, hoca bu aşamada öneriyi uygulamadı. Uyarıların bir kısmının makul, bir kısmının gereksiz olduğunu, bunlara çok takılmamak gerektiğini söyledi.

8. Kitap Sınıfı: Constructor Overloading ve `this(...)`

Aynı şey kitap için de yapıldı, fakat burada bir fark var: Bir kitabın yazarını bilmeyebiliriz. Bu yüzden constructor aşırı yüklendi (overloading):

- Birinci constructor yalnızca `id` ve `ad` alır.
- İkinci constructor `id`, `ad` ve `yazar` alır.

Birincide `yazar` alanına hiçbir şey yazılmaz, varsayılan olarak `null` kalır. İkincide ise `id` ve `ad` atamaları aynen tekrar edecektir. Hoca bu kod tekrarından kaçınmak için ikinci constructor'dan birincisini çağırmak istedi, ama söz dizimini bir süre aradı.

Soru: `this = new Kitap(...)` yazılabilir mi?

Cevap: Hayır. Derleyici "cannot assign to 'this' " hatası verir (javac ile denendi).

Soru: Doğrudan `Kitap(id, ad)` diye fonksiyon gibi yazsak olur mu?

Cevap: Hayır, öyle de olmaz. Derleyici bu adda bir metot arar ve "cannot find symbol" hatası verir (javac ile denendi).

Doğrusu `this`'i fonksiyon gibi çağırmaktır: `this(id, ad);`. Bu satır aynı sınıfın uygun parametreli constructor'ını çağırır, atamalar orada yapılır, sonra dönülür ve yalnızca `this.yazar = yazar;` kalır. Kod tekrarının önüne böyle geçilir.

> **Ek bilgi (derste anlatılmadı):** Klasik kurala göre `this(...)` constructor'ın ilk deyimi olmalıdır. `-source 21` ile önüne başka bir deyim yazılınca "flexible constructors is not supported in -source 21" hatası alındı. JDK 25'in varsayılan ayarıyla aynı kod derlendi ve çalıştı (javac ile denendi).

8.1 `kitapTanit` ve Yazılmış Metodu Yeniden Kullanmak

`kitapTanit`, kitabın ID ve adını yazar, sonra kitabın yazarı olup olmadığına bakar. Hoca önce tek satırda `this.yazar` değerini de yazdırmayı denedi. Fakat `this.yazar` bir referans olduğu için ekrana yine `Yazar@...` biçiminde bir değer gelir. Yazarın ID'sine ve adına ulaşmak gerekir. Bunun için `Yazar` sınıfında zaten çalışan bir metot var. Bir daha yazmaya gerek yoktur, çağırmak yeterlidir:

```
public class Kitap {
    private int id;
    private String ad;
    private Yazar yazar;

    public Kitap(int id, String ad) {
        this.id = id;
        this.ad = ad;
    }

    public Kitap(int id, String ad, Yazar yazar) {
        this(id, ad);
        this.yazar = yazar;
    }

    public void kitapTanit() {
        System.out.println("[ID:" + this.id + "--->AD:" + this.ad + "]");
        if (this.yazar != null)
            this.yazar.yazarTanit();
        else
            System.out.println("--->Yazar bulunamadi");
    }
}
```

Test sınıfında iki constructor'ın ikisi de kullanıldı. IDE, `new Kitap(` yazınca iki seçeneği de önerir:

```
Kitap k = new Kitap(1, "JAVA-1");
Kitap k2 = new Kitap(2, "JAVA-2", y1);
k.kitapTanit();
k2.kitapTanit();
```

Çıktı (javac ile denendi):

```
[ID:1--->AD:JAVA-1]
--->Yazar bulunamadi
[ID:2--->AD:JAVA-2]
[YAZAR ID:1--->YAZAR AD:ALI]
```

Bu koddaki iki şeyi hoca özellikle özetledi: Constructor overload edildi ve bilgi gizleme (information hiding) yapıldı. Yazarın bilgisi nesne oluşturulurken bir kez atanır, test uygulaması artık yazar bilgisi atayamaz. Hepsiburada örneğine dönersek, burası onun uygulamasıysa bunun gibi bir de kullanıcı uygulaması olacak. Kullanıcı buradan servis alacak, kitapları ve yazarları sorgulayacak.

Soru: Overload etmeden tek fonksiyonla yapamaz mıydık?

Cevap: Kitabın yazar bilgisine her zaman sahip olmayabiliriz. Tek constructor olsaydı yazar yokken parametre olarak `null` göndermek gerekirdi. İnsan bunu yazmayı unutabilir ve bu sorun çıkarır. Overloading bu tür durumların üstesinden gelmek için kullanılan bir mekanizmadır: Elde olan alanlar verilir, olmayanlar `null` kalır.

Soru: Kitabı tanıtırken `this.yazar.yazarTanit()` yerine `this.yazar.getId()` ile değeri çekip `println` içinde yazabilir miydik?

Cevap: Yazabilirdik. Ama zaten bir `yazarTanit` metodu var. `kitapTanit` onu çağırır, kod yeniden yazılmaz.

Soru: Java'da fonksiyon tanımlarına varsayılan değer verebiliyor muyuz? Swift'te `id: Int = 2` gibi yazılabiliyor, değer verilmezse o kullanılıyor.

Cevap: Hoca soruyu derste araştırdı. C#'ta parametreye = ile varsayılan değer verilir. Python'da da vardır: Ekrandaki örnekte `country` parametresi gönderilmezse `Norway` kullanılır, gönderilirse onun üzerine yazılır. Java'da böyle bir özellik yoktur, ona metot aşırı yüklemeyle benzer bir davranış kazandırılır. Olsaydı bu işi tek constructor ile halledebilirdik.

9. `static` Metotlar ve Alanlar

Bazı metotlar sınıfta olmazsa olmazdır, sürekli kullanılır ve nesne üretmeyi gerektirmez. Hoca, kitap örneği için çok uygun olmadığını söyleyerek şu metodu ekledi:

```
public static void f() {
    System.out.println("deneme");
}
```

Bu metot nesne üzerinden `k.f()` diye de çağrılır ve çalışır, ama IDE bunu bilerek önermez. Genelde sınıf üzerinden çağrılır: `Kitap.f();`. Sürekli kullanılan metotları statik tanımlamak, işi nesne üretmeden yapmayı sağlar.

> **Ders sonrası not:** Hocanın gösterdiği IDE davranışıyla uyumlu olarak javac da `-Xlint:static` ile `k.f()` için "static method should be qualified by type name" uyarısı verir (javac ile denendi).

9.1 `System.out` ve `Math`

Hoca `System.out.println` satırını örnek gösterdi: Biz bu satırda `System`'den de başka bir sınıftan da nesne üretmedik. `System` içindeki `out`, `PrintStream` türünde statik bir referanstır. `println` ise `out` nesnesi üzerinden ulaşılan bir metottur. Yani hoca önce "`PrintStream`'den de nesne üretmedik" dedi, sonra kendini düzeltti: Nesne `out` üzerinden üretilmiş durumda, biz ona ulaşıyoruz.

Daha rahat bir örnek `Math` sınıfıdır:

- `Math.abs(3.4)` çağrısında hiç değişken üretmedik. `abs` statiktir.
- `abs` aşırı yüklenmiştir. Hoca `int`, `float`, `double` ve "diğer şeyler" için sürümleri olduğunu gösterdi.
- `Math.E` ve `Math.PI` hem `static` hem `final` olan `double` alanlardır. Nesne üretmeden okunur ve `final` oldukları için değiştirilemez.

  Ders sonrası not: Yansımayla (reflection) bakıldığında `abs` için `int`, `long`, `float`, `double` sürümleri görüldü. `Math.E` ve `Math.PI` `public static final double`, `System.out` ise `public static final PrintStream` çıktı (javac ile denendi).

9.2 Statik Metottan Statik Olmayan Alana Erişim

`f()` içinde kitabın `id` alanını yazdırmak hata verir:

`non-static variable id cannot be referenced from a static context`

IDE'nin önerdiği çözüm: Ya değişken `static` yapılır ya da metodun `static` niteliği kaldırılır.

Hoca sonra metodun içinde statik bir değişken tanımlamayı denedi: `static int id = 1;`. Bu satır "illegal start of expression" hatası verdi. Hoca nedenini o an hatırlamadığını söyledi ve değişkeni sınıf düzeyine taşıdı.

> **Ders sonrası not:** Hatanın nedeni, Java'da C'deki gibi statik yerel değişken bulunmamasıdır. `static` alan sınıf düzeyinde tanımlanır. Metot içindeki satır aynı hatayı verdi (javac ile denendi).

9.3 Statik Alan Kaldığı Yerden Devam Eder

```
static int ids = 0;

public static void f() {
    System.out.println(ids);
    ids++;
}
```

Bu alan her çağrıda baştan başlamaz. İlk çağrıda `0` yazılır ve değer `1` olur, sonraki çağrıda `1` yazılır. Derste `f()` dört kez çağrıldığında çıktı `0, 1, 2, 3` oldu. Dersteki çağrılarla aynı sırada `k.f(); Kitap.f(); Kitap.f(); Kitap.f();` çalıştırılınca aynı çıktı alındı (javac ile denendi). Hoca bunun C'den bildiğimiz davranışla aynı olduğunu söyledi. `static`, `System.out` gibi yerlerde ya da nesne üretmeye gerek kalmayan durumlarda kullanılır.

10. `private` Set Metodu ve Şase Numarası Örneği

Set ve get metotları da `private` olabilir:

```
private void setId(int id) {
    this.id = id;
}
```

Bu metoda ne `Yazar`'dan ne `Test`'ten erişilebilir. Dışarıdan çağrılınca "setId(int) has private access" hatası alınır (javac ile denendi). Sınıfın içinde bir ID ataması gerekirse kullanılabilir. Aynı işi `this.id` ile de yapabiliriz.

Asıl önemli olan tasarım kararıdır. Hoca arabanın şase numarası örneğini hatırlattı: Nesne oluşturulur oluşturulmaz bir değer atanacak ve bu değer bir daha kesinlikle değiştirilmeyecekse, tek yol atamayı constructor ile yapmak ve set metodunu dışarıya kapatmaktır.

> **Hocanın vurgusu:** Bir kez atanıp değişmemesi gereken bir alanın set metodu dışarıya açık olursa bu, arka kapıdan dolaşmak gibi bir şey olur.

11. `finalize()`, Çöp Toplayıcı ve `@Override`

Hoca nesnenin işi biterken bir şey yapmayı sağlayan `finalize` metodunu gösterdi. Bu metot üst sınıflardan gelir. Hoca bunun büyük ihtimalle `Object` sınıfı olduğunu söyledi ve kontrol etti.

Kitap sınıfına şu eklendi:

```
@Override
public void finalize() {
    System.out.println(this.id + " nolu kitap silindi");
}
```

Bu kod hemen çalışmaz, çünkü nesnenin ne zaman silineceğine çöp toplayıcı (garbage collector) karar verir. Hoca test sınıfında bir nesneyi sahipsiz bıraktı:

```
k = k2;
```

Artık `k` ve `k2` aynı kitabı gösterir. ID'si 1 olan ilk kitabı gösteren referans kalmaz. Hocanın deyişiyle nesne "başıboş", zombi gibi kalır ve bir müddet sonra silinir. Silinirken de `finalize` çağrılır.

Deneme şu adımlarla ilerledi:

1. Yalnızca `k = k2;` ile çalıştırıldı. Program bitti, mesaj görülmedi. Hocanın deyişiyle "ona yetişemedik".
2. Bir öğrenci, nesne boşta kaldıktan sonra beklemeyi önerdi. `Thread.sleep(5000)` eklendi. Bu metot istisna fırlatabildiği için IDE'nin önerdiği bildirim de eklendi. Yine mesaj görülmedi. Hoca 5 saniyede silinmeyebileceğini söyledi.
3. Hoca `System.gc()` çağrısını buldu. Açıklamasında "runs the garbage collector" yazıyordu. Eklenip çalıştırılınca `1 nolu kitap silindi` yazısı görüldü.

```
k = k2;
System.gc();
Thread.sleep(5000);
```

> **Ders sonrası not:** Bu üç deneme JDK 25 ile tekrarlandı ve aynı sonuçlar alındı: Yalnız `k = k2;` ve `sleep` ile mesaj çıkmadı, `System.gc()` ve ardından bekleme ile `1 nolu kitap silindi` çıktı. Ancak bu davranış garanti değildir. `System.gc()` çağrısından hemen sonra program biterse mesaj çıkmadı, `-XX:+DisableExplicitGC` ile de mesaj çıkmadı (java ile denendi). Ayrıca `Object.finalize` metodu `@Deprecated(since="9", forRemoval=true)` olarak işaretlidir ve javac bunu "deprecated and marked for removal" uyarısıyla bildirir (javac ile denendi). Yeni kodda kullanılmamalıdır.
>
> **Ek bilgi (derste anlatılmadı):** Dosya ya da bağlantı gibi kaynakları serbest bırakmak için büyük olasılıkla `finalize` yerine `AutoCloseable` ve `try-with-resources` tercih edilir.

Soru: `@Override`'ı neden ekledik?

Cevap: Bu bir anotasyondur (annotation). `finalize`, `Object` sınıfında tanımlanmıştır ve kendine göre bir tanımı vardır. Biz onu kendimize göre değiştiriyoruz. Bu durumda oradaki metot değil, buradaki çalışır. `@Override` değiştirdiğimizi ifade eder. Hoca bunu eskiden zorunlu olduğunu, yeni sürümlerde zorunlu olmadığını söyledi.

> **Ders sonrası not:** Hoca eskiden zorunlu olduğunu söyledi; anotasyonsuz kod JDK 25'in varsayılan ayarıyla da `-source 8` ile de derlendi (javac ile denendi). Daha eski sürümlerde de büyük olasılıkla zorunlu değildi. Yararı başka yerdedir: Üst sınıfta karşılığı olmayan bir metoda yazılırsa derleyici "method does not override or implement a method from a supertype" hatası verir (javac ile denendi).
>
> **Ders sonrası not:** Hocanın `public` yazdığı metot üst sınıfta `protected`'tır (`protected void finalize() throws Throwable`, javac ile denendi). `public` yazılan sınıf JDK 25 ile derlendi, yani görünürlüğü artırarak ezmek geçerlidir.

12. Kütüphane Sınıfı: Kitap Dizisi, Ekleme, Listeleme

Hoca soruyu bir adım öteye taşıdı: Kullanıcı olarak kitap ve yazar oluşturup tanıtabiliyoruz. Şimdi bir de `Kutuphane` sınıfı ekleyelim. Kütüphanede kitaplar saklanacak, her kitabın da bir yazarı olacak. Üç sınıf iç içedir:

```
Kutuphane --- Kitap[] dizisi tutar ---> Kitap --- Yazar referansı tutar ---> Yazar
```

Kütüphanenin sınırlı bir kapasitesi olacak. Kitaplar bir dizide tutulacak. Kapasite sabit kalırken mevcut kitap sayısı değişeceği için onu ayrı bir alanda tutmak gerekir. Constructor'a ayrıca kitap dizisi vermeye gerek yoktur, yalnızca kapasite yeterlidir.

```
public class Kutuphane {
    int kapasite;
    Kitap[] kitaplar;
    int kitapSayisi;

    public Kutuphane(int kapasite) {
        kitapSayisi = 0;
        this.kapasite = kapasite;
        kitaplar = new Kitap[kapasite];
    }

    public boolean kitapEkle(Kitap k) {
        if (this.kitapSayisi < this.kapasite) {
            kitaplar[this.kitapSayisi] = k;
            kitapSayisi++;
            return true;
        }
        return false;
    }

    public void kutuphaneListele() {
        for (int i = 0; i < kitapSayisi; i++)
            this.kitaplar[i].kitapTanit();
    }
}
```

`new Kitap[kapasite]` kitap nesneleri üretmez, kapasite kadar referans tutacak bir yer açar. Yeni kütüphanede kitap sayısı sıfırdır. Hoca bu diziye "statik dizi" derken `static` anahtar sözcüğünü değil, boyutu sabit diziyi kastetti.

`kitapEkle` başarılı olup olmadığını göstermek için `boolean` döndürür. Yer varsa kitabı ekler ve `true` döner. Yer yoksa `false` döner.

Soru: Neden `this.kitapSayisi++` yazmadık?

Cevap: Yazabilirdik, ikisi aynı şey. Bu adda başka bir değişken olmadığı için `this` yazılmasa da olur. Hoca hızlı yazarken bazen `this`'i unuttuğunu söyledi.

Listelemede döngü `kapasite`'ye değil `kitapSayisi`'na kadar gider. Hoca sebebini açıkça söyledi: Kapasiteye kadar gidilirse boş alanlara da denk gelinebilir.

> **Ders sonrası not:** Boş hücre `null`'dur ve üzerinden metot çağrılınca `NullPointerException` alındı (javac ile denendi).

Bir kütüphane listelenince çağrılar iç içe ilerler:

1. Test sınıfı `kutuphaneListele()` metodunu çağırır.
2. Döngü her `i` için `kitaplar[i].kitapTanit()` metodunu çağırır.
3. `kitapTanit` kitabın ID ve adını yazar. Yazar varsa `yazarTanit()` metodunu çağırır.
4. `yazarTanit` yazarın bilgilerini yazar ve kontrol geri döner.

12.1 Kütüphaneyi Test Etmek

Test sınıfında yazar ve kitaplar aynen kaldı, `k3` ve `k4` eklendi. Önce kapasite 3 verildi:

```
Kutuphane kutuphane = new Kutuphane(3);
kutuphane.kitapEkle(k);
kutuphane.kitapEkle(k2);
kutuphane.kitapEkle(k3);
kutuphane.kitapEkle(k4);
kutuphane.kutuphaneListele();
```

Dört ekleme yapılmasına rağmen ekranda üç kitap göründü, çünkü dördüncü `kitapEkle` çağrısında $3 < 3$ şartı sağlanmaz ve metot `false` döner. Kapasite 4 yapılınca dördüncü kitap da geldi. Hesap adımları:

1. Kapasite 3: Dönüş değerleri `true true true false` olur, 3 kitap listelenir.
2. Kapasite 4: Dönüş değerleri `true true true true` olur, 4 kitap listelenir.

Her iki durum da javac ile denendi.

Soru: Bunlar dizinin sonuna mı eklenecek? İlk elemanlar boş kalıp bizimkiler arkaya mı gelecek?

Cevap: Hayır, baştan eklenir. Başta `kitapSayisi` 0'dır ve kapasiteden küçüktür. `kitaplar[0]` hücresine gelen kitap yazılır, sonra `kitapSayisi` 1 yapılır. Sonraki kitap için aynı kontrol yapılır ve `kitaplar[1]` kullanılır.

Hoca sonra dersin başındaki koda göndermede bulundu: Hangisi daha makul? Test eden kişi, ya da Hepsiburada veya Kitapyurdu gibi bir kurum, kitap eklemenin nasıl yapıldığını bilmek zorunda mı? Değil. Arka planda `LinkedList` mi normal dizi mi kullanıldığı onu ilgilendirmez. Ekliyorsa ve istenince listeliyorsa yeterlidir. Kullanıcıya bakan taraf ile arkadaki sarmalanmış kod bu şekilde ayrılır.

> **Hocanın vurgusu:** Kullanıcı açısından tamamen soyut bir durum vardır. Nasıl ekliyorsa ekliyor, beni ilgilendirmez.

13. Klavyeden Kapasite, Nesne Dizisi ve Bellek Düzeni

Hoca kütüphaneleri de kapsayan başka bir nesne olup olamayacağını sordu. Bir öğrenci okul önerdi. Bir okulda birden fazla kütüphane olabilir. Hoca bu yapıyı iç içe bohçalara benzetti ve kendi başına denenebileceğini söyledi.

İkinci kütüphanenin kapasitesi sabit değil, program çalışırken kullanıcıdan alınacak. Hoca burada C'deki gibi bir `malloc` olmadığını belirtti:

```
import java.util.Scanner;
// ...
Scanner sc = new Scanner(System.in);
System.out.println("Kutuphane kapasitesi giriniz:");
int n = sc.nextInt();
Kutuphane kutuphane2 = new Kutuphane(n);
```

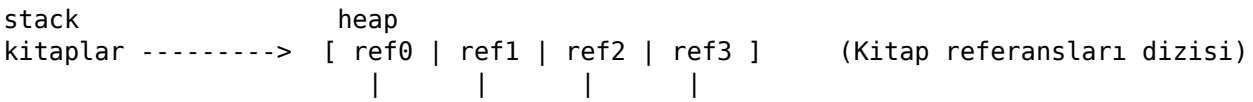
Klavyeden okumak için `Scanner` nesnesi `System.in` ile oluşturulur. `Scanner` için `java.util.Scanner` eklenmelidir.

Sonra kitaplar bir dizi üzerinden üretildi:

```
Kitap[] kitaplar = new Kitap[4];
for (int i = 0; i < 4; i++) {
    kitaplar[i] = new Kitap(i, "Java" + i, y1);
    kutuphane2.kitapEkle(kitaplar[i]);
}
kutuphane2.kutuphaneListele();
```

Hoca `new Kitap[4]`'ün bir constructor çağrısı olmadığına dikkat çekti: Bu satır kitap türünde dört referans içeren bir dizi oluşturur. Tek tek nesne üretmek için döngüde `new Kitap(...)` gerekir. Bu kod elle yazılan eklemelerin aynısını, referanslar üzerinden daha hızlı yapar. Kapasiteye 5 girildiğinde ekranda `Java0`, `Java1`, `Java2`, `Java3` listelendi (aynı çıktı javac ile denendi).

Bellek düzenini hoca şöyle anlattı: `k`, `k2`, `k3`, `k4` gibi değişkenleri stack bölgesinde düşünün. Dizi ise heap'te oluşur. `kitaplar` referansı stack'tedir ve dizinin başlangıç adresini tutar. Dizinin her hücresi de bir kitap nesnesini gösterir. Nesneler bellekte başka yerlerde, dağınık biçimde durur.



```
                    v      v      v      v
                  Kitap  Kitap  Kitap  Kitap       (bellekte dağınık nesneler)
```

Hoca bu yapıyı C'deki çift pointer'a benzetti, çünkü burada ikinci dereceden bir pointer vardır.

> **Hocanın vurgusu:** Nesne yönelimli programlama bir referanslar işidir. `=` ile bir nesnenin adresini başka bir yere bağlarız.

14. Toplu Ekleme: `kitapEkle(Kitap[])` ve Derste Çıkan Hata

Soru: `Kutuphane` sınıfına bu metodun kitap listesi alan bir sürümünü ekleyebilir miydik?

Cevap: Hoca tek tek eklemek yerine bir seferde eklemenin mümkün olduğunu söyledi ve metodu yazdı:

```
public boolean kitapEkle(Kitap[] k) {
    if (this.kapasite - this.kitapSayisi >= k.length) {
        this.kitaplar = k;
        return true;
    }
    return false;
}
```

Derleyici aynı isme itiraz etmedi. Hoca sebebini söyledi: İsimler aynı, ama parametreler farklı.

> **Ders sonrası not:** Öğrenci soruda "override" dedi. Aynı sınıfta parametresi farklı metot yazıldığı için bu, 8. bölümde görülen aşırı yüklemedir (overloading).

Şart şunu sorar: Boş yer, yani `kapasite - kitapSayisi`, gelen dizinin eleman sayısından büyük ya da ona eşit mi? Hoca dizilerin `length` diye bir değer sakladığını burada gösterdi. Ardından `this.kitaplar = k;` yazarken "birazcık problemli" olduğunu, olduğu gibi kopyalayamadığımızı, üstüne eklemek için ekstra iş gerektiğini de söyledi.

Test için ikinci bir dizi kuruldu:

```
Kitap[] kitaplar2 = new Kitap[4];
for (int i = 0; i < 4; i++) {
    kitaplar2[i] = new Kitap(i + 5, "Java" + i, y2);
}
kutuphane2.kitapEkle(kitaplar2);
kutuphane2.kutuphaneListele();
```

Çalıştırınca kapasite olarak 4 girildi ve iki listede de ID'ler `0, 1, 2, 3` göründü. Hoca önce `i + 5` değerinin hesaplanmamış olabileceğini düşündü, görebilmek için kitaplara ikinci bir yazar verdi ve debug başlattı. Sonunda "kapasite eksi kitap sayısı sıfır oluyor" diyerek sorunun şart olabileceğini gördü. Şartı kapatıp doğrudan `this.kitaplar = k;` çalıştırınca yeni dizinin kitapları, ID'leri 5'ten başlayarak listelendi.

Hesap adımları (python ile hesaplandı):

1. Verilen: `kapasite` 4, `kitapSayisi` 4 (döngü dört kitabı eklemiş), `k.length` 4.
2. Boş yer: $4 - 4 = 0$.
3. Şart: $0 \geq 4$ yanlıştır. Metot `false` döner, hiçbir kitap eklenmez.

> **Ders sonrası not:** Hocanın ilk yorumu olan "`i + 5` hesaplanmadı" doğru değildir. `i + 5` değerleri `5, 6, 7, 8` olur (python ile hesaplandı), ama bu kitaplar kütüphaneye hiç eklenmemiştir. Sebep şarttır. Kapasite 4 ile çalıştırıldığında metot `false` döndü ve ikinci listede yine `0, 1, 2, 3` göründü. Kapasite 5 ile de boş yer $5 - 4 = 1$ olduğu için aynı sonuç alındı (javac ile denendi).
>
> **Ders sonrası not:** Şart kaldırıldıktan sonraki `this.kitaplar = k;` satırı ekleme yapmaz. Kütüphanenin dizi referansını yeni diziye bağlar. Eski dört kitap kütüphaneden çıkar, `kitapSayisi` değişmez. Sadece bu örnekte `kitapSayisi` 4 ve yeni dizi de 4 elemanlı olduğu için liste düzgün görünür: ID'ler `5, 6, 7, 8` çıktı, sonda `kitaplar.length` 4 ve `kitapSayisi` 4 kaldı (javac ile denendi). Yeterli kapasite olduğunda kitaplar mevcutların sonuna tek tek eklenmelidir:

```java
public boolean kitapEkle(Kitap[] k) {
    if (this.kapasite - this.kitapSayisi < k.length)
        return false;
    for (Kitap x : k)
        kitapEkle(x);
    return true;
}
```

Bu sürüm kapasite 8 ile çalıştırıldı: Boş yer $8 - 4 = 4$ olduğu için şart sağlanır. Sekiz kitap listelendi, ID'ler `0..3` ve `5..8`, sonda `kitapSayisi` 8 oldu. Kapasite 4 ile `false` döndü ve liste değişmedi (javac ile denendi; hesap python ile yapıldı).

Hoca da aynı sonuca işaret etti: Sabit boyutlu dizide en son kaldığımız yerden devam etmek gerekiyorsa bir `for` döngüsü yazmak zorundayız. `LinkedList` ya da `ArrayList` gibi yapılar görülünce gelen listeyi mevcut listenin sonuna "append" etmek mümkün olacak. Bizim yapımız sınırlı bir dizi olduğu için biraz daha sıkıntılıdır.

> **Ders sonrası not:** Hocanın "append" sözü işlemi anlatıyor. `ArrayList` üzerinde `append` çağrısı "cannot find symbol" hatası verdi, `addAll` çalıştı (javac ile denendi).

15. Tekrar Alıştırması: Birden Fazla Yazar

Hoca dersi bitirirken bu konuların tekrarı için bir değişiklik önerdi: Bir kitabın birden fazla yazarı olsun. Bu durumda:

- `Kitap` içindeki yazar alanı artık bir dizi tutar.
- `kitapTanit` içinde, kitabın kaç yazarı varsa o kadar dönen bir döngü çalışır. Çünkü yazar sayısı bir de olabilir, iki de.

Hoca bu değişikliğin bugünkü konuları tekrar etmek için faydalı olacağını söyledi.

16. Dizinin Sonu, `length` ve Dönüş Değerini Kullanmamak

Soru: `kutuphaneListele` gibi metotlarda kullanılan for-each yapısı nasıl çalışıyor?

Cevap: Hoca konuya daha sonra geleceklerini söyledi. For-each, dizi şeklindeki bir yapıyı oluşturan elemanları tek tek alır. İstersek toplarız, istersek ekrana yazdırırız.

Soru: Dizide dönerken sonuna geldiğimizi nasıl anlıyoruz? `null` görünce mi çıkmalıyız?

Cevap: Dizinin kaç elemanlı olduğu bilinir. `kitaplar.length` bu değeri verir, parantezsiz yazılır. IDE bunu `public final int length` olarak gösteriyor. Hoca bunun dinamik bağlama sayesinde geldiğini, `Object` sınıfında ya da bir yerde tanımlı olabileceğini ve dizi oluşturulurken bir kez atanıp değiştirilemediğini söyledi. Benzetme için bir `final int` alan örneği verdi: Bu alan constructor'da atanır, sonra değiştirilemez. `kitapSayisi++` gibi bir satır o durumda geçersiz olur ("cannot assign a value to final variable", javac ile denendi).

> **Ders sonrası not:** Hoca `length` için "dinamik bağlama", "objede olabilir" ve "derleyici tarafından tutuluyor" dedi. Doğrusu: `length` dizi nesnesine özgüdür. `Object` türündeki değişkende `o.length` yazılınca "cannot find symbol" hatası alındı. Atama denenince "cannot assign a value to final variable length" hatası alındı. Kapasite klavyeden okunan `n` ile `new Kitap[n]` yapıldığında `length`, girilen değeri verdi. Yani değer derleme sırasında değil, dizi oluşturulurken belirlenir (javac ile denendi).
>
> **Ders sonrası not:** Hoca konuşma sırasında `length`'in "objeleri sayabildiğini" de söyledi. `length`, içindeki nesne sayısını değil dizinin boyutunu verir: 5 hücreli dizinin 3 hücresi doldurulurken `length` 5 çıktı (javac ile denendi). Kütüphanede `kitapSayisi` alanı bu yüzden ayrıca tutulur.

Soru: `boolean` döndüren bir metodun değerini kullanmadan bırakabiliyor muyuz?

Cevap: Evet. `kitapEkle` değer döndürür, ama bunu alıp bir `if` içinde kullanmak zorunda değiliz. İstenirse "kitap eklenemedi" diye bir mesaj verilebilir. Hoca bunu `kitapTanit` metodunu

geçici olarak `int` döndürür yapıp denedi: Yalnızca IDE uyarısı geldi, kod çalıştı. Dönüş değerini kullanmayan çağrılar derleniyor ve çalışıyor (javac ile denendi). Hoca, Java'da mı C++'ta mı olduğunu hatırlamadığı, dönen değerin alınmasını isteyen bir uyarı olduğunu da söyledi.

17. Sayısal Tipler Arasında Atama

Hoca dersin sonunda slayttaki tipleri kısaca geçti: `long` 8 bayt, `short` 2 bayt, `byte` 1 bayttır (`Long.BYTES`, `Short.BYTES`, `Byte.BYTES` değerleri 8, 2 ve 1 çıktı, javac ile denendi). Bu tipler arasında kural şudur: Küçükler büyüklere dönüşür, büyükler küçüklere dönüşmez.

```
byte b = 1;
short s = 3;
s = b;       // geçerli: küçükten büyüğe
int c = b;   // geçerli
c = s;       // geçerli
// b = s;    // derleme hatası
// s = c;    // derleme hatası
```

Hatalı satırlarda derleyici "incompatible types: possible lossy conversion from short to byte" ve "... from int to short" iletilerini verir (javac ile denendi). `float` ile `double` için de aynısı geçerlidir: `float` değer `double` değişkene atanır, tersi olmaz.

Soru: `s`'nin değeri sıfır olsa da olmuyor değil mi?

Cevap: Olmaz, bu değerle ilgili değildir. Bir tipin içindeki değişken diğerine aktarılıyor ve dil kayıpsız işlem yapmak istiyor. Hoca bunlar değer tipleri olduğu için burada "deep copy" yapıldığını, değerin bir yere kopyalandığını söyledi.

> **Ders sonrası not:** Değişkenler için cevap doğrudur, `short s = 0; byte b = s;` derlenmedi. İstisna, derleme zamanında bilinen sabitlerdir: `final short s = 3; byte b = s;` derlendi ve `3` yazdı (javac ile denendi). Ayrıca primitif kopyalama için "deep copy" terimi büyük olasılıkla yerinde değildir. Burada yapılan düz değer kopyalamadır, terim nesne kopyalama bağlamında kullanılır.

Hoca `char` ve `boolean` tiplerini de andı, `boolean`'ın C'de olmadığını söyledi ve listeye `String`'i de ekledi.

> **Ders sonrası not:** C99 standardında `_Bool` tipi vardır, `gcc -std=c99` ile derlendi. `String` ise primitif değildir, hocanın "bu değişkenlerin hepsi primitiftir" genellemesine girmez (javac ile denendi).

Dersin Toparlaması

Bu derste kod, alanları dışarıdan atanan basit sınıflardan başlayıp soyutlamaya doğru adım adım dönüştü. `new` ile üretilen nesnelerin referans tipli olduğu, `==` işlecinin referans karşılaştırdığı ve alanların varsayılan değerleri görüldü. Ardından alanlar `private` yapıldı, değerler constructor ile verildi, `this` ile alan ve parametre ayrıldı. `Kitap` constructor'ı aşırı yüklendi ve `this(...)` ile kod tekrarı önlendi. `static` metot ve alanlar, `private` set metodu ve `finalize` ile çöp toplayıcı gösterildi. `Kutuphane` sınıfı bu yapıların üstüne kuruldu: Kullanıcı artık yalnızca `kitapEkle` ve `kutuphaneListele` çağırıyor, kitapların nasıl saklandığını bilmiyor.

Hoca bir sonraki derste sarmalayıcı sınıflara (wrapper class) ve onların özel metotlarına geçileceğini, `toString` ve `equals` metotlarının bunlar arasında olduğunu söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.